

جامعة دمشق

كلية الهندسة المعلوماتية — قسم هندسة البرمجيات

تقرير مشروع مادة البرمجة المتوازية

الفصل الدراسي 2026

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

(High-Performance E-Commerce Backend Engine)

ShopFlow

إعداد الطلاب:

عبدالرحمن الكفري

عبدالرحمن الضماد

أيهم الحايك

جلال كاخي

التقنيات المستخدمة: Spring Boot 3.4.3 • Java 17 • PostgreSQL • Redis

Nginx • Docker Compose • Prometheus • Grafana

فهرس المحتويات

1. مقدمة

2. المتطلب الأول — حماية البيانات المشتركة من التضارب

2.1 القفل المتفائل (Optimistic Locking)

2.2 القفل المتشائم (Pessimistic Locking)

2.3 نمط الاستراتيجية (Strategy Pattern)

3. المتطلب الثاني — إدارة الموارد الحاسوبية

3.1 إدارة اتصالات قاعدة البيانات (HikariCP)

3.2 إدارة اتصالات Redis (Lettuce Pool)

3.3 إدارة مجمعات الخيوط (Thread Pool Executors)

4. المتطلب الثالث — المعالجة غير المتزامنة

4.1 نمط الحدث/المستمع (Event/Listener Pattern)

4.2 إرسال البريد الإلكتروني بشكل غير متزامن

4.3 إبطال الرموز عبر Redis

5. المتطلب الرابع — معالجة البيانات الضخمة على دفعات

5.1 المهمة المجدولة (Scheduled Job)

5.2 المعالجة على شكل دفعات (Chunk Processing)

6. المتطلب الخامس — توزيع الأحمال

6.1 البنية التحتية (Docker Compose + Nginx)

6.2 استراتيجية التوزيع (Round-Robin via DNS)

6.3 تسجيل الطلبات عبر AOP

6.4 المراقبة باستخدام Prometheus و Grafana

7. اختبار الضغط — سلامة البيانات تحت التزامن العالي

8. التخزين المؤقت والقياس المرجعي وتحليل عنق الزجاجة

8.1 فكرة التخزين المؤقت في Redis

8.2 تصميم تجربة القياس (قبل / بعد)

8.3 النتائج وتحليل عنق الزجاجة

9. المعاملات الذرية (ACID Transactions)

10. ملخص الملفات والتقنيات

1. مقدمة

يهدف مشروع **ShopFlow** إلى بناء نظام خلفي (Backend) متكامل لمنظومة تجارة إلكترونية، مع التركيز على المتطلبات غير الوظيفية المتعلقة بالبرمجة المتوازية والأداء العالي. يُغطي هذا التقرير المتطلبات الخمسة الأولى من متطلبات المشروع، مع شرح مفصّل للحلول المعمارية والكود المصدري المستخدم لتحقيق كل مطلب.

تم بناء النظام باستخدام إطار عمل **Spring Boot 3.4.3** مع **Java 17**، ويعتمد على **PostgreSQL 16** كقاعدة بيانات رئيسية، و **Redis 7** لإدارة الرموز والتخزين المؤقت، و **Nginx** كموزع أحمال، و **Docker Compose** لتنسيق الحاويات. كما تم دمج **Prometheus** و **Grafana** لمراقبة الأداء في الزمن الحقيقي.

2. المتطلب الأول — حماية البيانات المشتركة من التضارب

(Concurrent Access & Data Integrity)

يتطلب النظام آلية تسمح لعدة مستخدمين بتعديل نفس المورد (مثل كمية المنتج في المخزون) دون حدوث تضارب أو فقدان بيانات. تم تطبيق نهج مزدوج يجمع بين **القفل المتفائل للعمليات العادية** و**القفل المتشائم للعمليات ذات التضارب العالي** (مثل التخفيضات السريعة).

2.1 القفل المتفائل (Optimistic Locking)

يعتمد القفل المتفائل على افتراض أن التضارب نادر الحدوث. يُضاف حقل `Version` لكيان المنتج، يزداد تلقائياً مع كل عملية تحديث. إذا حاول خيطان تعديل نفس السجل في الوقت ذاته، يفشل الثاني بخطأ `OptimisticLockException`.

كيان المنتج مع حقل الإصدار:

src/main/java/.../product/entity/Product.java

```
@Entity
@Table(name = "products")
public class Product extends BaseEntity {

    private String name;
    private int stockQuantity;
    private BigDecimal price;

    @Version
    @Column(nullable = false)
    @Builder.Default
    private Long version = 0L;
}
```

عند تنفيذ عملية التحديث، يُولّد JPA استعلام SQL يتضمن شرط الإصدار:

```
UPDATE products
SET stock_quantity = ?, version = version + 1
WHERE id = ? AND version = ?
```

إذا لم يتطابق الإصدار (أي أن خيطاً آخر عدّل السجل)، لا يُحدّث أي سطر ويُطرح استثناء. تتعامل استراتيجية الشراء المتفائلة مع هذا الاستثناء عبر إعادة

المحاولة:

استراتيجية الشراء بالقفل المتفائل:

src/main/java/.../product/strategy/OptimisticPurchaseStrategy.java

```
@Component("optimisticPurchase")
public class OptimisticPurchaseStrategy implements
PurchaseStrategy {

    private static final int MAX_RETRIES = 3;

    @Override
    @Transactional
    public Product purchase(UUID productId, int quantity) {
        int attempt = 0;
        while (attempt < MAX_RETRIES) {
            try {
                return attemptPurchase(productId, quantity);
            } catch (ObjectOptimisticLockingFailureException
ex) {
                attempt++;
                log.warn("Optimistic lock conflict - attempt
{}/{}", attempt, MAX_RETRIES);
            }
            throw new ConflictException("Could not complete
purchase after " + MAX_RETRIES + " retries");
        }

        private Product attemptPurchase(UUID productId, int
quantity) {
            Product product = productRepository.findById(productId)
                .orElseThrow(() -> new
ResourceNotFoundException("Product not found"));
            if (product.getStockQuantity() < quantity) {
                throw new InsufficientStockException("Not enough
stock");
            }
            product.setStockQuantity(product.getStockQuantity() -
quantity);
            return productRepository.save(product);
        }
    }
}
```

2.2 القفل المتشائم (Pessimistic Locking)

في حالة التخفيضات السريعة (Flash Sales)، يكون التضارب شبه مؤكد مع عشرات المستخدمين يتنافسون على كمية محدودة. هنا يُستخدم القفل

المتشائم الذي يقفل السجل على مستوى قاعدة البيانات فوراً باستخدام
SELECT ... FOR UPDATE

الاستعلام مع القفل المتشائم:

src/main/java/.../flashsale/repository/FlashSaleRepository.java

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT f FROM FlashSale f WHERE f.id = :id")
Optional<FlashSale> findByIdWithLock(@Param("id") UUID id);
```

يُترجم هذا إلى استعلام SQL:

```
SELECT * FROM flash_sales WHERE id = ? FOR UPDATE
```

أي خيط آخر يحاول قراءة نفس السجل سيُحظر تلقائياً حتى ينتهي الخيط
الأول من المعاملة (COMMIT أو ROLLBACK).

استراتيجية الشراء بالقفل المتشائم:

src/main/java/.../product/strategy/PessimisticPurchaseStrategy.java

```
@Component("pessimisticPurchase")
public class PessimisticPurchaseStrategy implements
PurchaseStrategy {

    @Override
    @Transactional
    public Product purchase(UUID flashSaleId, int quantity) {
        FlashSale flashSale =
flashSaleRepository.findByIdWithLock(flashSaleId)
                .orElseThrow(() -> new
ResourceNotFoundException("Flash sale not found"));

        if (flashSale.getRemainingQuantity() < quantity) {
            throw new InsufficientStockException("Flash sale
sold out");
        }

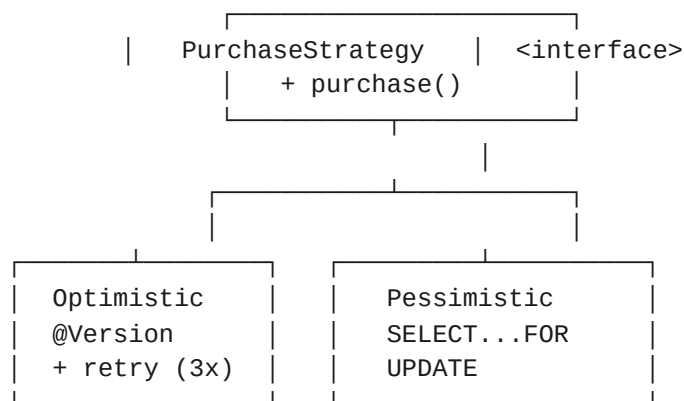
        flashSale.setRemainingQuantity(flashSale.getRemainingQuantity()
- quantity);
        flashSaleRepository.save(flashSale);
        return flashSale.getProduct();
    }
}
```

2.3 نمط الاستراتيجية (Strategy Pattern)

تم تطبيق نمط التصميم Strategy لاختيار آلية القفل المناسبة حسب نوع الطلب. في خدمة الطلبات، يتم تفويض اختيار القفل إلى الاستراتيجية المناسبة:

```
// الشراء العادي → قفل متفائل
Product product =
optimisticStrategy.purchase(itemReq.getProductId(),
itemReq.getQuantity());

// تخفيضات سريعة → قفل متشائم
Product product = pessimisticStrategy.purchase(flashSaleId,
quantity);
```



نتائج الاختبار العملي — القفل المتفائل والمتشائم

تم اختبار آلية القفل عبر سكريبتات Bash تُرسل طلبات متزامنة لمحاكاة سيناريوهات الاستخدام الحقيقي. تم اختبار الأداء مع القفل وبدون القفل لتوضيح أهمية آلية الحماية.

أولاً: اختبار القفل المتفائل (مع تفعيل @Version وآلية إعادة المحاولة)

إرسال 10 طلبات شراء متزامنة لمنتج iPhone 16 Pro (المخزون = 50). النتيجة: المخزون انخفض بشكل صحيح، مع ظهور رسائل `OptimisticLockingFailureException` في السجلات عند حدوث تضارب، وتم إعادة المحاولة تلقائياً.

لإثبات أهمية آليات القفل، تم إزالة @Version من كيان المنتج وإزالة @Lock من الاستعلام وآلية إعادة المحاولة. النتائج أظهرت فقدان بيانات كارثي:

```
May 18 17:33 | 155.7 B/s 1.251 B/s | en |
abdu... | 2026-05-18 14:33:09 [9b5b645a41b1] INFO [http-nio-8080-exec-10] o.e.s.s
order.service.OrderService - Order 2ac1126c-e6fc-47a5-bb5f-bd3a15596148 placed for user 397c43
e2-9c75-4f03-9719-6d0952951de8 - async invoice triggered
app-1 | 2026-05-18 14:33:09 [9b5b645a41b1] INFO [order-async-2] o.e.s.order.ev
ent.OrderEventListener - [ASYNC] Handling OrderPlacedEvent for order 2ac1126c-e6fc-47a5-bb5f-b
d3a15596148 in thread order-async-2
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-6] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: ecdd20b8c3dc] --> POST /api/orders (handler: OrderCo
ntroller.placeOrder(...))
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-4] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=50 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-6] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=50 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
app-1 | 2026-05-18 14:33:09 [9b5b645a41b1] INFO [order-async-4] o.e.s.order.ev
ent.OrderEventListener - [ASYNC] Handling OrderPlacedEvent for order 0994b58e-b7cb-40a8-a94c-1
f8575cc5e0 in thread order-async-4
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-5] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=50 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-4] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=50 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
app-2 | 2026-05-18 14:33:09 [ecdd20b8c3dc] INFO [http-nio-8080-exec-6] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=50 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-2] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 47ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-2] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 47ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-9] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 53ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-9] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 53ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-7] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 60ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-7] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: 9b5b645a41b1] <-- POST /api/orders completed in 60ms
app-1 | 2026-05-18 14:33:10 [9b5b645a41b1] INFO [http-nio-8080-exec-6] o.e.s.p
.s.OptimisticPurchaseStrategy - NO LOCK - read stock=49 for product 273f1610-9a64-4a03-b8ca-c2
9d036df6c7
abdu... | OPTIMISTIC LOCKING TEST
Product: iPhone 16 Pro (stock=50)
Sending 10 concurrent purchase requests...
>>> Stock BEFORE:
iPhone 16 Pro: stock=50
Request 8: HTTP 201
Request 10: HTTP 201
Request 5: HTTP 201
Request 1: HTTP 201
Request 4: HTTP 201
Request 2: HTTP 201
Request 3: HTTP 201
Request 6: HTTP 201
Request 7: HTTP 201
Request 9: HTTP 201
>>> Stock AFTER:
iPhone 16 Pro: stock=49
>>> Check logs for 'Optimistic lock conflict' messages:
docker compose logs app 2-81 | grep -i 'lock conflict'
abdu... |
```

شكل 3: بدون @Version — إرسال 10 طلبات متزامنة وجميعها نجحت (HTTP 201) لكن المخزون انخفض من 50 إلى 49 فقط بدلاً من 40! (فقدان 9 عمليات تحديث)

في السجلات نلاحظ أن جميع الخيوط قرأت نفس القيمة stock=50 في الوقت ذاته، وكل خيط طرح منها 1 وحفظ 49. الكتابة الأخيرة هي التي سادت (Last Write Wins)، مما أدى لفقدان 9 عمليات شراء.

```
May 18 17:34 | 12.01 K/s 1.165 K/s | en |
abdu... | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-5] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: ecdd20b8c3dc] <-- POST /api/orders/flash-sale comple
ted in 52ms
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-7] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: ecdd20b8c3dc] <-- POST /api/orders/flash-sale comple
ted in 69ms
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [order-async-4] o.e.s.invoice.
service.InvoiceService - Invoice INV-20260518143415-A9808C57 generated for order a9808c57-ed25
-44aa-9940-e31d3d2bfdf0
shopflow-nginx | 172.22.0.1 - [18/May/2026:14:34:15 +0000] "POST /api/orders/flash-sale
HTTP/1.1" 201 upstream: 172.22.0.4:8080 response time: 0.007s
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-8] o.e.s.p
.s.PessimisticPurchaseStrategy - Flash sale purchase - remaining: 8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-2] o.e.s.p
.s.PessimisticPurchaseStrategy - Flash sale purchase - remaining: 8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-11] o.e.s.p
.s.PessimisticPurchaseStrategy - Flash sale purchase - remaining: 8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-3] o.e.s.o
rder.service.OrderService - Flash sale order 5f59e573-7bc5-4b1c-a542-0ca00b4803b3 placed for u
ser 397c43e2-8c75-4f03-9719-6d0952951de8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-8] o.e.s.o
rder.service.OrderService - Flash sale order c65f5f5c-7cf8-4096-a37d-2500eca2e716 placed for u
ser 397c43e2-8c75-4f03-9719-6d0952951de8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [order-async-5] o.e.s.invoice.
service.InvoiceService - Invoice INV-20260518143415-0C53742E generated for order 0c53742e-9fcb
-4629-b122-9298d18dd774
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-6] o.e.s.s
.config.RequestLoggingAspect - [INSTANCE: ecdd20b8c3dc] <-- POST /api/orders/flash-sale comple
ted in 66ms
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-11] o.e.s.
order.service.OrderService - Flash sale order 2c66731f-82c3-470d-8ca2-6dbdd15bca94 placed for
user 397c43e2-8c75-4f03-9719-6d0952951de8
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [order-async-1] o.e.s.invoice.
service.InvoiceService - Invoice INV-20260518143415-DBFF6D85 generated for order dbff6d85-cf37
-4161-bb2a-05d0d2aeca058
shopflow-nginx | 172.22.0.1 - [18/May/2026:14:34:15 +0000] "POST /api/orders/flash-sale
HTTP/1.1" 201 upstream: 172.22.0.4:8080 response time: 0.090s
app-2 | 2026-05-18 14:34:15 [ecdd20b8c3dc] INFO [http-nio-8080-exec-9] o.e.s.p
.s.PessimisticPurchaseStrategy - Flash sale purchase - remaining: 8
abdu... | PESSIMISTIC LOCKING TEST
Flash Sale: Samsung Galaxy S25 (qty=10)
Sending 15 concurrent requests...
Expected: 10 succeed, 5 rejected
>>> Flash Sale BEFORE:
Traceback (most recent call last):
File "<string>", line 1, in <module>
KeyError: 'data'
Request 2: SUCCESS (HTTP 201)
Request 5: SUCCESS (HTTP 201)
Request 8: SUCCESS (HTTP 201)
Request 7: SUCCESS (HTTP 201)
Request 6: SUCCESS (HTTP 201)
Request 9: SUCCESS (HTTP 201)
Request 3: SUCCESS (HTTP 201)
Request 4: SUCCESS (HTTP 201)
Request 10: SUCCESS (HTTP 201)
Request 15: SUCCESS (HTTP 201)
Request 13: SUCCESS (HTTP 201)
Request 1: SUCCESS (HTTP 201)
Request 11: SUCCESS (HTTP 201)
Request 12: SUCCESS (HTTP 201)
Request 14: SUCCESS (HTTP 201)
>>> Flash Sale AFTER:
Traceback (most recent call last):
File "<string>", line 1, in <module>
KeyError: 'data'
>>> No data was lost - pessimistic lock serialized all 15 requ
ests.
>>> Check logs: docker compose logs app 2-81 | grep -i 'flash\
|pessimistic'
abdu... |
```

شكل 4: اختبار القفل المتشائم مع التفعيل — جميع 15 طلب تمت معالجتها بشكل متسلسل عبر SELECT...FOR UPDATE

3. المتطلب الثاني — إدارة الموارد الحاسوبية

(Resource Management & Capacity Control)

يتطلب النظام التحكم في عدد العمليات المتوازية لمنع استنزاف الموارد (انهيار) أو تقليلها بشكل يبطئ الاستجابة. تم تطبيق هذا المتطلب على ثلاثة مستويات: اتصالات قاعدة البيانات، اتصالات Redis، ومجمعات الخيوط.

3.1 إدارة اتصالات قاعدة البيانات (HikariCP)

HikariCP هو مجمع اتصالات عالي الأداء يأتي مدمجاً مع Spring Boot. تم ضبطه للسماح بحد أقصى 10 اتصالات متزامنة لقاعدة البيانات:

src/main/resources/application.yml

```
spring:
  datasource:
    hikari:
      pool-name: HikariPool-AuthDB
      maximum-pool-size: 10      # الحد الأقصى للاتصالات
      minimum-idle: 5           # الحد الأدنى للاتصالات الجاهزة
      idle-timeout: 300000      # دقائق - إغلاق الاتصالات 5
      connection-timeout: 30000 # ثانية - فشل سريع بدلاً من 30
      max-lifetime: 1800000     # دقيقة - إعادة تدوير 30
      leak-detection-threshold: 60000 # تحذير إذا بقي اتصال مفتوحاً أكثر من 60 ثانية
```

المعامل	القيمة	الهدف
maximum-pool-size	10	منع إغراق PostgreSQL بعدد كبير من الاتصالات
minimum-idle	5	ضمان جاهزية 5 اتصالات دائماً لتقليل زمن الاستجابة
connection-timeout	30 ثانية	فشل سريع بدلاً من حجز الخيط إلى ما لا نهاية
leak-detection-threshold	60 ثانية	كشف تسريب الاتصالات (نسيان إغلاقها)

3.2 إدارة اتصالات Redis (Lettuce Pool)

يُستخدم Redis لإبطال رموز JWT وتخزين البيانات المؤقتة. تم ضبط مجمع Lettuce لتحديد عدد الاتصالات المتزامنة:

```
spring:
  data:
    redis:
      lettuce:
        pool:
          max-active: 10      # الحد الأقصى للاتصالات المتزامنة
          max-idle: 5         # اتصالات خاملة جاهزة
          min-idle: 2         # اتصالات دافئة دائماً
          max-wait: 2000ms    # فشل بعد ثانيتين إذا لم يتوفر
# اتصال
```

3.3 إدارة مجمعات الخيوط (Thread Pool Executors)

تم تعريف مجمعي خيوط منفصلين لعزل الموارد بين المهام المختلفة ومنع تأثير نوع من المهام على نوع آخر:

src/main/java/.../shared/config/AsyncConfig.java

```

@Configuration
@EnableAsync
@EnableScheduling
public class AsyncConfig {

    // مجمع خيوط لمعالجة أحداث الطلبات (فواتير، إشعارات)
    @Bean(name = "orderExecutor")
    public Executor orderExecutor() {
        ThreadPoolTaskExecutor executor = new
ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);           // خيوط أساسية 5
        executor.setMaxPoolSize(20);          // يتوسع حتى 20 خيط
        // تحت الضغط
        executor.setQueueCapacity(100);        // طا بور يستوعب 100
        // مهمة
        executor.setThreadNamePrefix("order-async-");
        executor.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy() //
تدهور أنيق
        );
        executor.initialize();
        return executor;
    }

    // مجمع خيوط مخصص للمهام الدفعية (معزول عن حركة الطلبات)
    @Bean(name = "batchExecutor")
    public Executor batchExecutor() {
        ThreadPoolTaskExecutor executor = new
ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(10);
        executor.setThreadNamePrefix("batch-job-");
        executor.setRejectedExecutionHandler(
            new ThreadPoolExecutor.CallerRunsPolicy()
        );
        executor.initialize();
        return executor;
    }
}

```

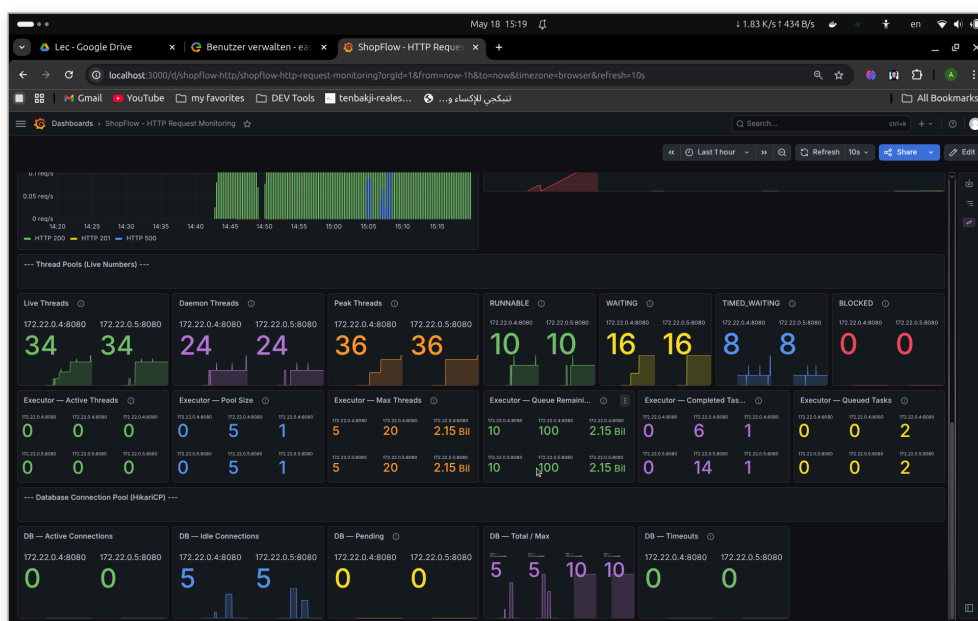
ملاحظة معمارية — CallerRunsPolicy:

عند امتلاء الطابور والوصول للحد الأقصى من الخيوط، بدلاً من رفض المهمة (وفقدانها)، يقوم الخيط المُستدعي بتنفيذها بنفسه. هذا يُبطئ قبول المهام الجديدة بشكل طبيعي (Back-pressure) دون فقدان أي بيانات.

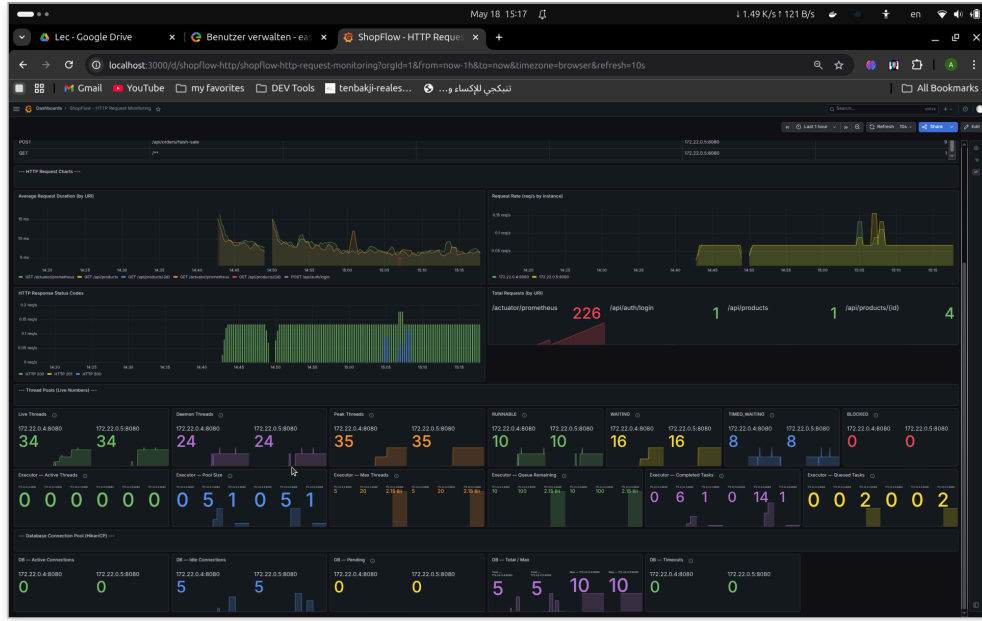
Application Thread Pools	
orderExecutor Core: 5 Max: 20 Queue: 100	batchExecutor Core: 2 Max: 5 Queue: 10
Invoice generation Email sending Notifications	Daily sales report Token cleanup

نتائج المراقبة العملية — حالة الموارد في الزمن الحقيقي

تم رصد حالة جميع الموارد عبر لوحة مراقبة Grafana أثناء تنفيذ اختبارات التزامن. تُظهر اللقطات التالية حالة مجمعات الخيوط واتصالات قاعدة البيانات لكلا النسختين:



شكل 5: حالة مجمعات الخيوط — Live Threads: 34 لكل نسخة، Executor Pool Size: 5، Peak: 36، Queue Remaining لكل مجمع خيوط، واتصالات HikariCP (5 خاملة، 10 كحد أقصى)



شكل 6: مقاييس مفضلة — رموز حالة (200, 201, 500), إجمالي الطلبات لكل مسار, مع إحصائيات مجمعات الخيوط (Executor Active/Pool Size/Max/Queue) وقاعدة البيانات (Active/Idle/Pending/Total Connections)

4. المتطلب الثالث — المعالجة غير المتزامنة

(Asynchronous Queues)

يتطلب نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل إصدار الفواتير أو إرسال الإشعارات) خارج المسار الرئيسي للطلب. بذلك يحصل المستخدم على استجابة فورية بينما تُنفَّذ المهام الثقيلة في الخلفية.

4.1 نمط الحدث/المستمع (Event/Listener Pattern)

عند إتمام طلب شراء، يُنشر حدث `OrderPlacedEvent` عبر نظام أحداث Spring. يلتقطه مستمع غير متزامن يعمل في خيط منفصل:

نشر الحدث عند إتمام الطلب:

`src/main/java/.../order/service/OrderService.java`

```
@Transactional
public Order placeOrder(UUID userId, PlaceOrderRequest request)
{
    // ... إنشاء الطلب وتحديث المخزون ...
    order = orderRepository.save(order);

    // نشر الحدث - المستمع يعالج الفاتورة بشكل غير متزامن
    eventPublisher.publishEvent(new OrderPlacedEvent(this,
order));
    log.info("Order placed - async invoice triggered");

    return order; // الاستجابة تُرسل فوراً قبل إنشاء الفاتورة
}
```

المستمع غير المتزامن:

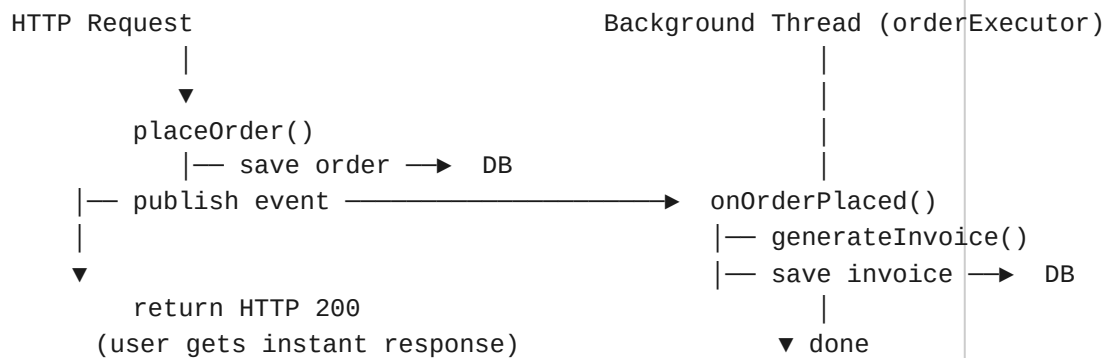
`src/main/java/.../order/event/OrderEventListener.java`

```

@Component
public class OrderEventListener {

    @Async("orderExecutor") // يعمل في مجمع خيوط الطلبات
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
        Order order = event.getOrder();
        log.info("[ASYNC] Handling OrderPlacedEvent in thread
    {"},
                Thread.currentThread().getName());
        try {
            invoiceService.generateInvoice(order);
            log.info("[ASYNC] Invoice generated for order {"},
order.getId());
        } catch (Exception ex) {
            log.error("[ASYNC] Failed to generate invoice for
order {"}, order.getId());
        }
    }
}

```



4.2 إرسال البريد الإلكتروني بشكل غير متزامن

جميع عمليات إرسال البريد الإلكتروني (التحقق من الحساب، إعادة تعيين كلمة المرور) تتم بشكل غير متزامن حتى لا يُحظر الخيط الرئيسي بانتظار خادم :SMTP

src/main/java/.../auth/service/EmailService.java

```

@Service
public class EmailService {

    @Async    // يعمل في خيط منفصل
    public void sendVerificationEmail(String to, String token)
    {
        String verificationUrl = baseUrl + "/api/auth/verify-email?token=" + token;
        sendHtmlEmail(to, "Verify your email address", body);
    }

    @Async
    public void sendPasswordResetEmail(String to, String token)
    {
        String resetUrl = baseUrl + "/api/auth/reset-password?token=" + token;
        sendHtmlEmail(to, "Reset your password", body);
    }
}

```

4.3 إبطال الرموز عبر Redis

عند تسجيل الخروج، يتم إدراج رمز JWT في القائمة السوداء عبر Redis مع مدة انتهاء تلقائية (TTL)، مما يمنع إعادة استخدام الرمز:

```

public void blacklist(String jti, long remainingMs) {
    String key = "jwt:blacklist:" + jti;
    redisTemplate.opsForValue().set(key, "1",
    Duration.ofMillis(remainingMs));
    // Redis المفتاح تلقائياً بعد انتهاء المدة
}

```

5. المتطلب الرابع — معالجة البيانات الضخمة على دفعات

(Batch Processing)

يتطلب برمجة وظيفة خلفية تقوم بجدد المبيعات اليومية ومعالجتها كدفعات (Chunks) لزيادة سرعة الأداء. تم تحقيق ذلك عبر مهمة مجدولة تستخدم الترقيم الصفحي (Pagination) لتقسيم البيانات.

5.1 المهمة المجدولة (Scheduled Job)

يتم تشغيل تقرير المبيعات اليومية تلقائياً عند منتصف الليل بتوقيت UTC، في مجمع خيوط مخصص ومعزول عن حركة الطلبات:

src/main/java/.../report/scheduler/DailySalesJob.java

```
@Component
public class DailySalesJob {

    @Async("batchExecutor") // مجمع خيوط مخصص (2 أساسي، 5 حد أقصى)
    @Scheduled(cron = "0 0 0 * * *", zone = "UTC") // كل يوم عند منتصف الليل
    public void runDailyReport() {
        LocalDate yesterday = LocalDate.now().minusDays(1);
        log.info("[BATCH] Starting daily sales report for {} in thread {}",
                yesterday, Thread.currentThread().getName());
        reportService.generateReportForDate(yesterday);
    }
}
```

5.2 المعالجة على شكل دفعات (Chunk Processing)

بدلاً من تحميل جميع الطلبات في الذاكرة دفعة واحدة (مما قد يسبب OutOfMemoryError)، تتم المعالجة على دفعات بحجم 500 سجل باستخدام Pageable:

src/main/java/.../report/service/DailySalesReportService.java

```

@Service
public class DailySalesReportService {

    private static final int CHUNK_SIZE = 500;

    @Transactional
    public void generateReportForDate(LocalDate date) {
        // التحقق من عدم وجود تقرير مُسبق (idempotency)
        if (reportRepository.existsByReportDate(date)) {
            log.info("Report for {} already exists - skipping",
date);
            return;
        }

        DailySalesReport report = DailySalesReport.builder()
            .reportDate(date)
            .status(ReportStatus.PROCESSING)
            .build();
        report = reportRepository.save(report);

        int page = 0;
        int totalOrders = 0;
        BigDecimal totalRevenue = BigDecimal.ZERO;
        Page<Order> chunk;

        try {
            do {
                // سجل في كل مرة CHUNK_SIZE جلب
                chunk =
orderRepository.findByStatusAndCreatedAtBetween(
                    OrderStatus.CONFIRMED, from, to,
                    PageRequest.of(page, CHUNK_SIZE)
                );

                for (Order order : chunk.getContent()) {
                    totalRevenue =
totalRevenue.add(order.getTotalAmount());
                }
                totalOrders += chunk.getNumberOfElements();

                log.info("Batch chunk {} processed - {}
orders", page, chunk.getNumberOfElements());
                page++;

            } while (chunk.hasNext());

            report.setTotalOrders(totalOrders);
            report.setTotalRevenue(totalRevenue);
            report.setStatus(ReportStatus.COMPLETED);
            reportRepository.save(report);

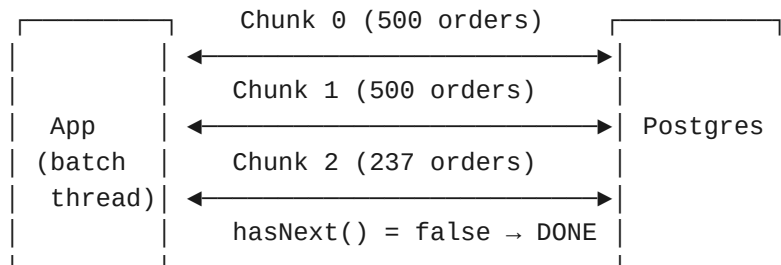
        } catch (Exception ex) {
            report.setStatus(ReportStatus.FAILED);

```

```

        reportRepository.save(report);
        log.error("Daily report for {} failed: {}", date,
ex.getMessage());
    }
}
}

```



Memory at any point: max 500 Order objects (not 1237)

ملاحظة — تنظيف الرموز المنتهية:

بالإضافة لتقرير المبيعات، توجد مهمة مجدولة أخرى تعمل كل ساعة لحذف رموز التحديث المنتهية والملغاة من قاعدة البيانات:

```

@Scheduled(fixedRate = 3600000) // كل ساعة
@Transactional
public void cleanupStaleTokens() {

    refreshTokenRepository.deleteExpiredTokens(Instant.now());

    refreshTokenRepository.deleteRevokedTokensBefore(now.minus(
24, ChronoUnit.HOURS));
    log.info("Token cleanup completed");
}

```

نتائج الاختبار العملي — المعالجة على دفعات مقابل التحميل الكامل

تم اختبار المعالجة على دفعات بحجم 50 سجل لـ 144 طلب. تُظهر السجلات أن كلتا النسختين عالجتا 3 دفعات (Chunk 0: 50 طلب، Chunk 1: 50 طلب، Chunk 2: 44 طلب) بشكل متوازٍ:

```
shopflow-grafana | logger=dashboard-service t=2026-05-18T13:18:23.719802096Z level=Info msg="No last resource version found, starting from scratch" orgID=
shopflow-grafana | logger=dashboard-service t=2026-05-18T13:18:53.723727035Z level=Info msg="No last resource version found, starting from scratch" orgID=
app-2 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
026-05-17 in thread batch-job-1 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
app-2 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 0 processed - 50 orders in this
chunk | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 0 processed - 50 orders in this
app-2 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 1 processed - 50 orders in this
chunk | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 1 processed - 50 orders in this
app-2 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
chunk | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
app-2 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Daily report for 2026-05-17 completed - 144
orders, revenue: 38476.97 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
chunk | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Daily report for 2026-05-17 completed - 144
orders, revenue: 38476.97 | 2026-05-18 13:19:07 [04f6aa01eae] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
app-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
026-05-17 in thread batch-job-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 0 processed - 50 orders in this
chunk | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 0 processed - 50 orders in this
app-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 1 processed - 50 orders in this
chunk | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 1 processed - 50 orders in this
app-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
chunk | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
app-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Daily report for 2026-05-17 completed - 144
orders, revenue: 38476.97 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
app-1 | 2026-05-18 13:19:07 [f4f4e66380f8] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Batch chunk 2 processed - 44 orders in this
```

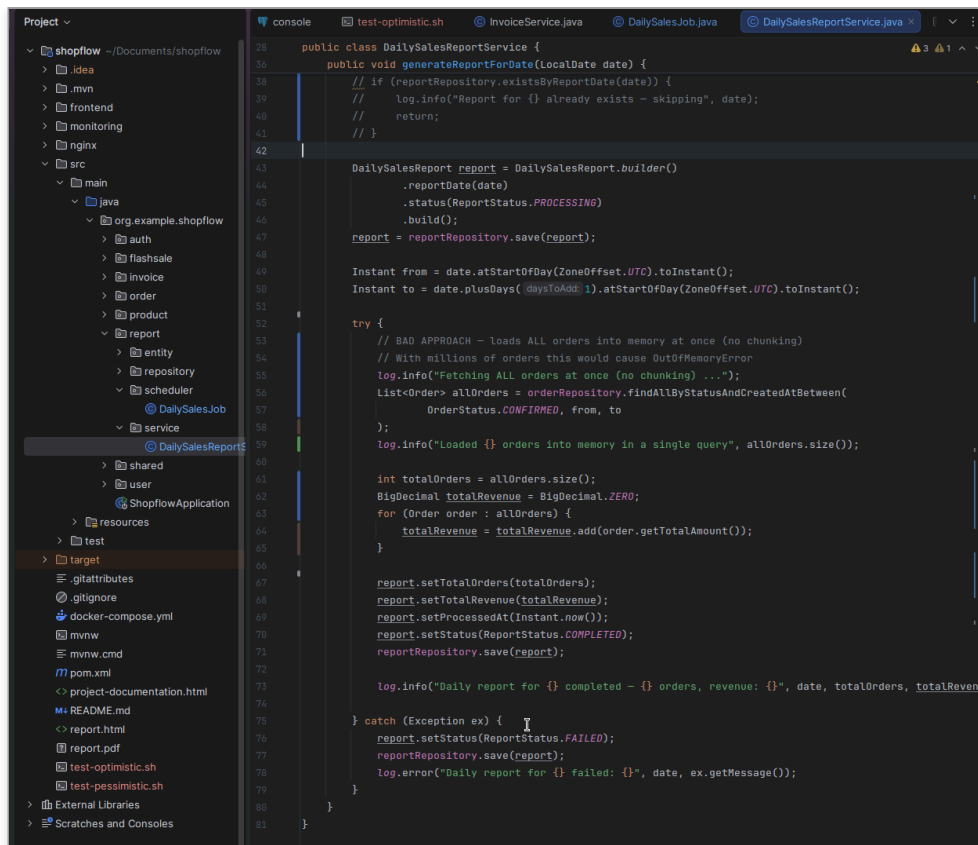
شكل 7: المعالجة على دفعات — كلنا النسختين (app-1, app-2) تعالجان 144 طلب على 3 دفعات (50+50+44) في خيط batch-job-1

مقارنة: التحميل الكامل بدون دفعات (النهج غير المحسّن)

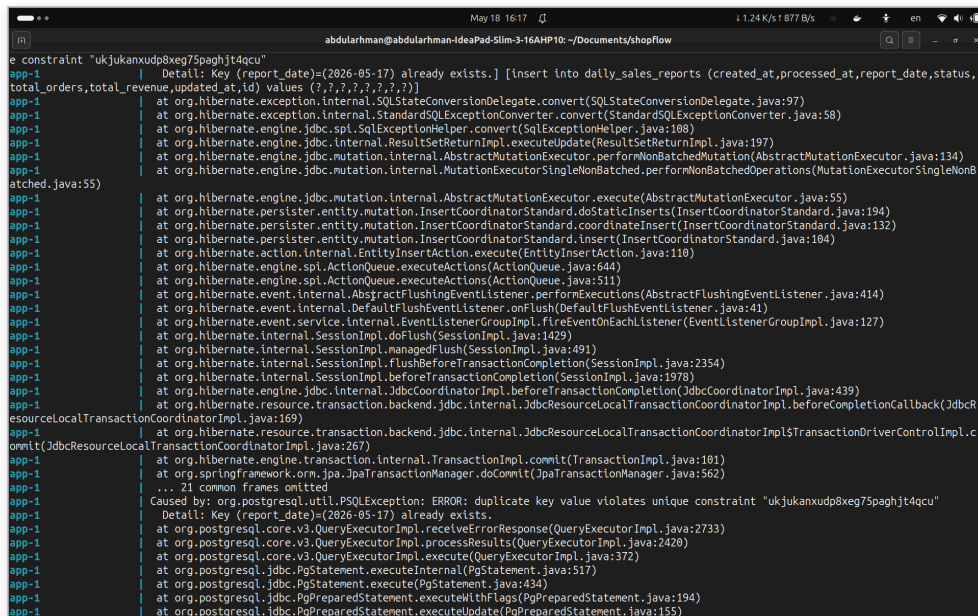
لإثبات أهمية نهج الدفعات، تم تعديل الكود لتحميل جميع الطلبات دفعة واحدة في الذاكرة. تُظهر السجلات الرسالة "Fetching ALL orders at once (no chunking)" — في بيئة إنتاجية مع ملايين السجلات، سيؤدي هذا النهج إلى OutOfMemoryError:

```
app-2 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [restartedMain] o.s.b.d.a.OptionalLiveReloadServer - LiveReload server is running on port 3572
app-2 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [restartedMain] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with con
text path '/' | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [restartedMain] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with con
app-2 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [restartedMain] o.e.shopflow.ShopFlowApplication - Started ShopFlowApplication in 13.765 secon
ds (process running for 14.642) | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [restartedMain] o.e.shopflow.ShopFlowApplication - Started ShopFlowApplication in 13.765 secon
app-2 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
026-05-17 in thread batch-job-1 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
app-2 | 2026-05-18 13:56:09 [eaeae0b265f5] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Fetching ALL orders at once (no chunking) ..
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.s.b.d.a.OptionalLiveReloadServer - LiveReload server is running on port 3572
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.s.b.d.a.OptionalLiveReloadServer - LiveReload server is running on port 3572
app-2 | 2026-05-18 13:56:10 [eaeae0b265f5] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Fetching ALL orders at once (no chunking) ..
app-2 | 2026-05-18 13:56:10 [eaeae0b265f5] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Fetching ALL orders at once (no chunking) ..
app-2 | 2026-05-18 13:56:10 [eaeae0b265f5] INFO [scheduling-1] o.e.s.s.s.TokenCleanupScheduler - Token cleanup completed
app-2 | 2026-05-18 13:56:10 [eaeae0b265f5] INFO [scheduling-1] o.e.s.s.s.TokenCleanupScheduler - Token cleanup completed
app-2 | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with con
text path '/' | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with con
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.e.shopflow.ShopFlowApplication - Started ShopFlowApplication in 14.149 secon
ds (process running for 14.767) | 2026-05-18 13:56:10 [a782837decdd] INFO [restartedMain] o.e.shopflow.ShopFlowApplication - Started ShopFlowApplication in 14.149 secon
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
026-05-17 in thread batch-job-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [batch-job-1] o.e.s.report.scheduler.DailySalesJob - [BATCH] Starting daily sales report for 2
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Fetching ALL orders at once (no chunking) ..
app-1 | 2026-05-18 13:56:10 [a782837decdd] INFO [batch-job-1] o.e.s.r.s.DailySalesReportService - Fetching ALL orders at once (no chunking) ..
```

شكل 8: النهج غير المحسّن — تحميل جميع الطلبات في الذاكرة دفعة واحدة (Fetching ALL orders at once)



شكل 9: كود *DailySalesReportService* بدون دفعات — يستخدم *findAllByStatusAndCreatedAtBetween* لتحميل كل شيء مرة واحدة



شكل 10: خطأ *duplicate key* — عندما تحاول كلتا النسختين إنشاء التقرير في الوقت ذاته، تفشل إحداهما بسبب القيد الفريد على *report_date* (سلوك متوقع ومحمي)

6. المتطلب الخامس — توزيع الأحمال

(Load Distribution)

يتطلب محاكاة توزيع الطلبات على أكثر من خادم مع توضيح وتبرير استراتيجية التوزيع المستخدمة. تم تحقيق ذلك عبر Nginx كموزع أحمال أمام نسختين من تطبيق Spring Boot، مُدارة بواسطة Docker Compose.

6.1 البنية التحتية (Docker Compose + Nginx)

يتم تشغيل نسختين من التطبيق عبر أمر التوسيع في Docker Compose:

```
docker compose up --scale app=2
```

هذا يُنشئ حاويتين متطابقتين (app-1 و app-2) خلف شبكة Docker واحدة. يتعامل Nginx مع جميع الطلبات الواردة على المنفذ 80 ويوزعها بين النسختين:

(المقتطفات الأساسية) docker-compose.yml

```
services:
  app:
    image: shopflow:latest
    # ضروري للتوسيع - container_name
    # يتعامل مع الوصول الخارجي Nginx - ports
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app
```

6.2 استراتيجية التوزيع (Round-Robin via Docker DNS)

تم اعتماد استراتيجية Round-Robin (التناوب الدوري) لتوزيع الطلبات بالتساوي بين النسختين. يعتمد التطبيق على نظام DNS الداخلي لـ Docker الذي

يُعيد عناوين IP مختلفة عند كل استعلام لاسم الخدمة app.

النقطة الحرجة: يجب استخدام متغير في proxy_pass لإجبار Nginx على إعادة الاستعلام عن DNS في كل طلب (بدون ذلك يُخزّن Nginx العنوان مرة واحدة عند الإقلاع ويُرسَل جميع الطلبات لنسخة واحدة):

nginx/nginx.conf

```
server {
    listen 80;
    server_name localhost;

    # إعادة الاستعلام كل 5 ثواني - Docker الداخلي لـ DNS مُحلل
    resolver 127.0.0.11 valid=5s ipv6=off;

    # حجب نقاط الوصول الداخلية من الخارج
    location /actuator {
        deny all;
        return 403;
    }

    location / {
        # في كل طلب DNS المتغير يُجبر إعادة استعلام
        set $backend http://app:8080;
        proxy_pass $backend;

        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

لماذا Round-Robin وليس استراتيجية أخرى؟

بما أن نسختي التطبيق متطابقتان في الموارد والإعدادات، فإن التوزيع بالتساوي هو الخيار الأمثل. استراتيجيات مثل Least Connections أو IP Hash تُضيف تعقيداً دون فائدة عندما تكون الخوادم متماثلة.

6.3 تسجيل الطلبات عبر AOP

لإثبات أن التوزيع يعمل فعلياً، تم تطبيق مفهوم البرمجة الموجهة للجوانب (AOP) لاعتراض كل طلب HTTP وتسجيل مُعرّف النسخة التي عالجتها:

src/main/java/.../shared/config/RequestLoggingAspect.java

```

@Aspect
@Component
@Slf4j
public class RequestLoggingAspect {

    @Value("${HOSTNAME:single}")
    private String instanceId;

    @Around("@within(org.springframework.web.bind.annotation.RestController)")
    public Object logRequest(ProceedingJoinPoint joinPoint)
    throws Throwable {
        HttpServletRequest request =
            ((ServletRequestAttributes)
                RequestContextHolder.getRequestAttributes()).getRequest();

        String method = request.getMethod();
        String uri = request.getRequestURI();

        log.info("[INSTANCE: {}] --> {} {} (handler: {})",
            instanceId, method, uri,
            joinPoint.getSignature().toShortString());

        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;

        log.info("[INSTANCE: {}] <-- {} {} completed in {}ms",
            instanceId, method, uri, duration);

        return result;
    }
}

```

عند تشغيل `docker compose logs app` وإرسال عدة طلبات،
تظهر السجلات بالتناوب بين النسختين:

```

app-1 | [INSTANCE: aa087c423ab6] --> GET /api/products
completed in 13ms
app-2 | [INSTANCE: 32433b18c8ce] --> GET /api/products
completed in 13ms
app-1 | [INSTANCE: aa087c423ab6] --> GET /api/products
completed in 2ms
app-2 | [INSTANCE: 32433b18c8ce] --> GET /api/products
completed in 3ms

```

6.4 المراقبة باستخدام Prometheus و Grafana

لمراقبة أداء الطلبات ومدتها في الزمن الحقيقي، تم دمج **Prometheus** لجمع المقاييس و**Grafana** لعرضها بصرياً. يستخدم التطبيق **Spring Boot** **Actuator** مع **Micrometer** لتصدير المقاييس بتنسيق Prometheus:

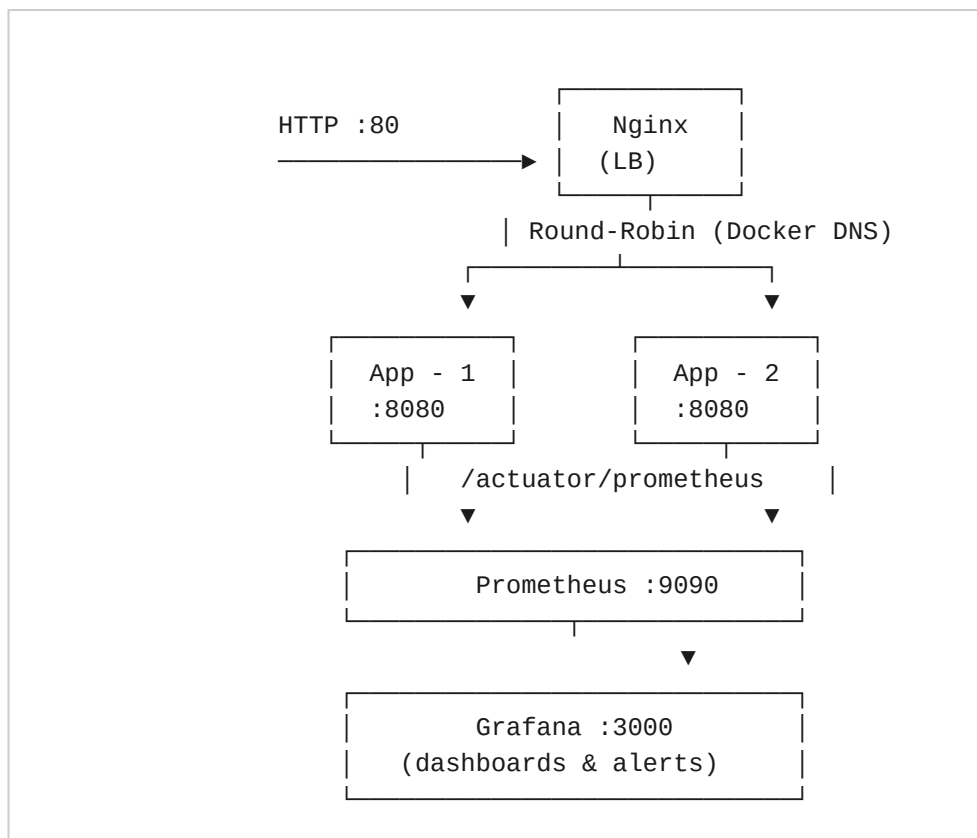
```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  metrics:
    tags:
      application: shopflow
    distribution:
      percentiles-histogram:
        http.server.requests: true
      percentiles:
        http.server.requests: 0.5,0.95,0.99
```

يقوم Prometheus بجمع المقاييس من كلتا النسختين تلقائياً عبر اكتشاف DNS:

```
scrape_configs:
  - job_name: 'shopflow'
    metrics_path: '/actuator/prometheus'
    dns_sd_configs:
      - names: ['app']
        type: 'A'
        port: 8080
```

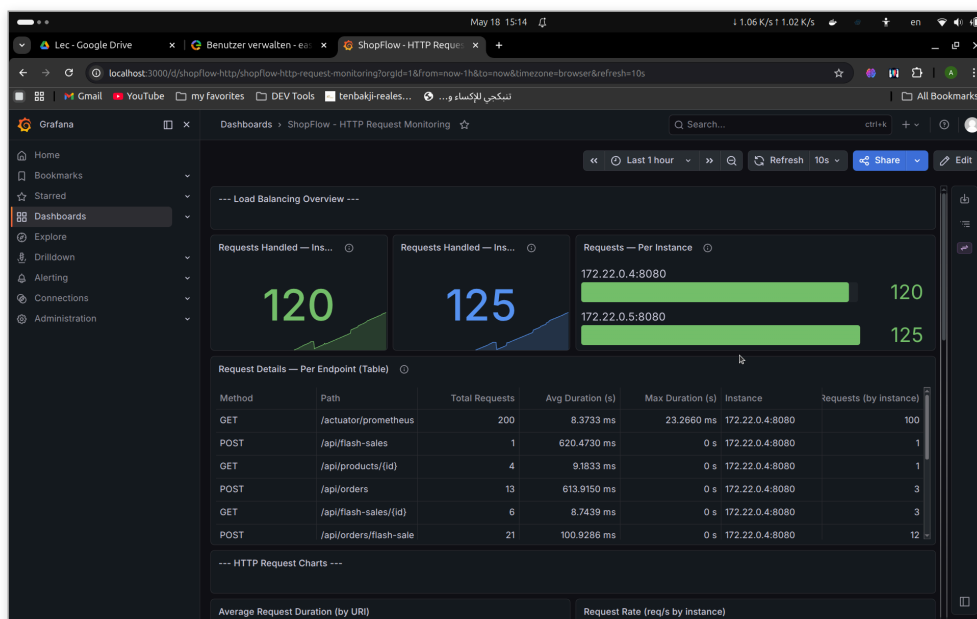
تم إعداد لوحة مراقبة Grafana مُسبقة التهيئة تعرض:

- عدد الطلبات المعالجة لكل نسخة (لإثبات توزيع الحمل)
- جدول تفصيلي بكل مسار: المدة المتوسطة، المدة القصوى، عدد الطلبات
- حالة الخيوط (نشطة، خاملة، محجوبة) كأرقام مباشرة
- حالة مجمع اتصالات قاعدة البيانات (HikariCP)
- معدل الطلبات في الثانية لكل نسخة ورموز حالة HTTP

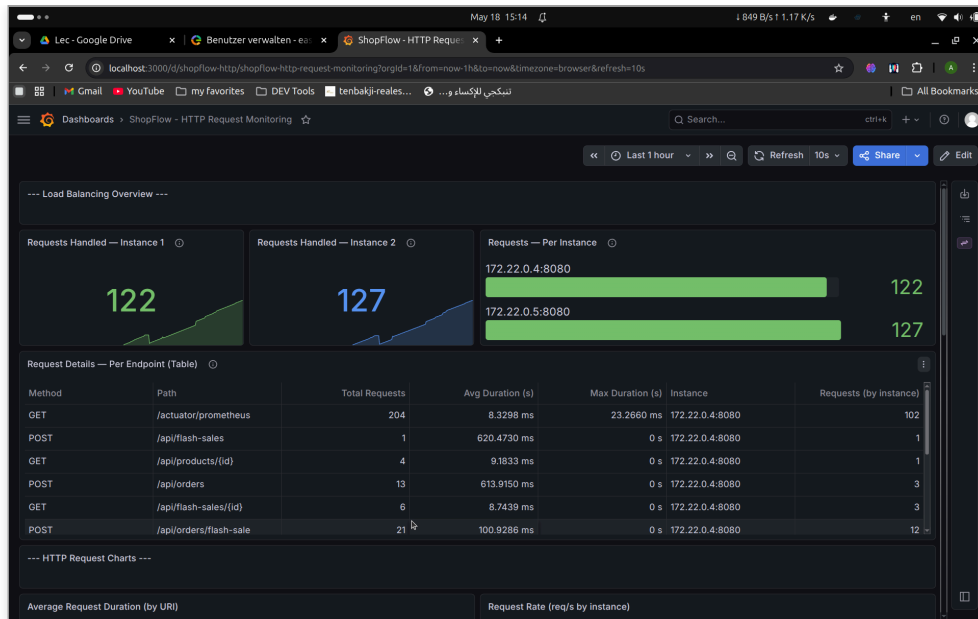


نتائج المراقبة العملية — لوحة Grafana

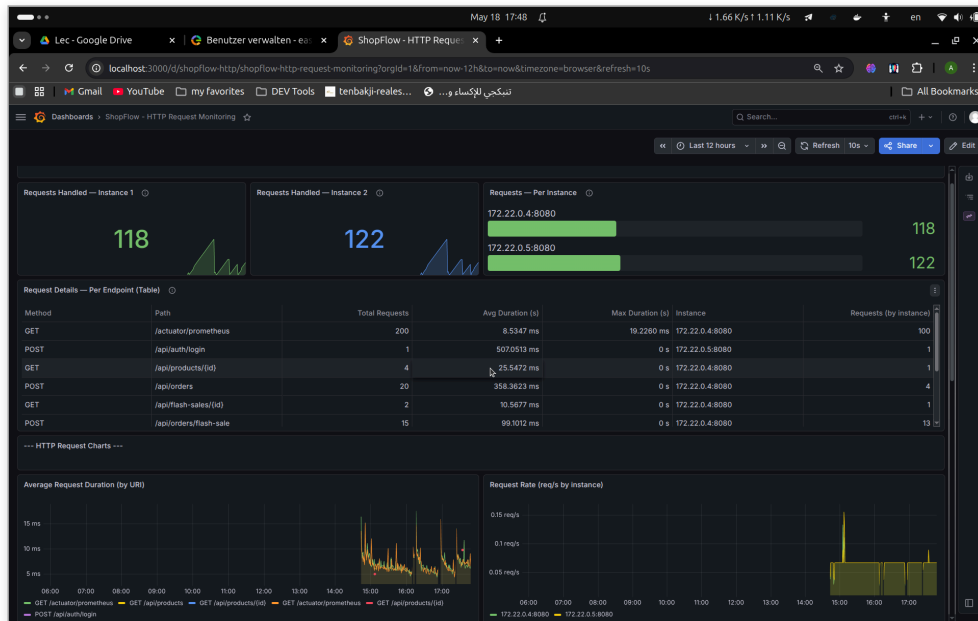
تُظهر لوحة المراقبة توزيع الأحمال بشكل متساوٍ بين النسختين، مع تفاصيل كل مسار API ومدة الاستجابة:



شكل 11: نظرة عامة على توزيع الأحمال — Instance 1: 120 طلب، Instance 2: 125 طلب (توزيع شبه متساوٍ عبر Round-Robin)



شكل 12: جدول تفصيلي بكل نقطة نهاية — المدة المتوسطة، المدة القصوى، النسخة المُعالجة، وعدد الطلبات لكل نسخة



شكل 13: اللوحة النهائية — 118/122 طلب لكل نسخة، مع رسوم بيانية لمدة الاستجابة ومعدل الطلبات في الثانية على مدى 12 ساعة

7. اختبار الضغط (Stress Testing) — إثبات سلامة البيانات تحت التزامن العالي

لإثبات أن النظام يصمد تحت التزامن العالي دون فقدان أي بيانات، تم إجراء اختبار ضغط باستخدام **Apache JMeter** (عبر صورة Docker الرسمية) يحاكي 100 مستخدم يرسلون طلب إنشاء طلب شراء (POST /api/orders) في اللحظة نفسها دفعة واحدة.

7.1 تصميم الاختبار

تم تشغيل الاختبار عبر السكريبت `loadtest/bench.sh` بالإعدادات التالية:

```
WRITE_USERS=100 LOOPS=1 RAMPUP=0 ./loadtest/bench.sh
integrity
```

- 100 مستخدم متزامن (`WRITE_USERS=100`) — كل مستخدم يرسل طلباً واحداً فقط (`LOOPS=1`)، أي 100 طلب بالضبط.
- بدء فوري دفعة واحدة (`RAMPUP=0`) — تنطلق كل الخيوط في اللحظة نفسها لتوليد أقصى تنافس (Thundering Herd).
- كتالوج صغير (20 منتجاً) — لتركيز التنافس على عدد قليل من الصفوف وإجبار آلية القفل على العمل.

7.2 آلية الحماية تحت الاختبار

عند تزامن عدّة طلبات على المنتج نفسه، يرفض القفل المتفائل (`Version@ + إعادة المحاولة`) الطلبات المتضاربة بدلاً من إفساد البيانات. الطلب المرفوض هو رفض نظيف (لا يغيّر المخزون)، وليس فقداناً للبيانات. بعد انتهاء الاختبار تم التحقق من قاعدة البيانات للتأكد من تحقق الثابت التالي:

```
عدد الطلبات) = (مجموع كميات عناصر الطلبات) = (المخزون المُباع)
(المؤكدة)
```

أي أن كل وحدة نقصت من المخزون تقابلها بالضبط عنصر طلب واحد وطلب مؤكّد واحد — لا بيع زائد، ولا فقدان أو تكرار لأي وحدة.

نتيجة الاختبار العملي — 100 طلب متزامن

```
DATA-INTEGRITY (Req 9) - 20 products · writes only · 100 users × 1 = 100 requests

>> resetting stack (wiping volumes)...
[+] Running 12/12
✓ Network shopflow_shopflow-network Created0.1s
✓ Volume shopflow_grafana-data Created0.0s
✓ Volume shopflow_postgres-data Created0.0s
✓ Volume shopflow_redis-data Created0.0s
✓ Volume shopflow_prometheus-data Created0.0s
✓ Container shopflow-postgres Healthy5.8s
✓ Container shopflow-redis Healthy5.8s
✓ Container shopflow-app-2 Started5.9s
✓ Container shopflow-app-1 Started6.0s
✓ Container shopflow-prometheus Started6.2s
✓ Container shopflow-nginx Started6.1s
✓ Container shopflow-grafana Started6.3s
>> waiting for app to seed 20 products...
>> ready (~15s, 20 products)
>> stress run (writes only)...
summary = 102 in 00:00:03 = 39.2/s Avg: 479 Min: 42 Max: 1084 Err: 16 (15.69%)

RESULT - Req 9 data integrity

stock sold (seeded - now) : 85
units sold (order_items) : 85
confirmed orders : 85

✓ PASS - every unit sold maps to exactly one order item and one confirmed order.
No overselling, no lost or duplicated stock under concurrent load.

Raw results: loadtest/results/integrity.jtl
```

شكل 14: 100 مستخدم يرسلون الطلب نفسه في اللحظة ذاتها — الثابت محقق تماماً (المُباع = الكميات = الطلبات المؤكدة) ✓ بدون أي بيع زائد أو فقدان بيانات

8. التخزين المؤقت (Caching) والقياس المرجعي وتحليل عنق الزجاجة

يجمع هذا المتطلب بين ثلاثة أمور مترابطة في تجربة واحدة: التخزين المؤقت (المتطلب 6) كتحسين، واختبار الضغط (المتطلب 9) كأداة قياس، والمقارنة قبل/بعد (المتطلب 10) لتحليل لعنق الزجاجة. كما تراقب لوحات Grafana (المتطلب 8) حالة النظام أثناء التجربة.

8.1 فكرة التخزين المؤقت في Redis

مسار قراءة المنتجات (GET /api/products) هو المسار الأكثر استخداماً. بدلاً من الوصول إلى PostgreSQL في كل طلب، يتم تخزين النتيجة في Redis وخدمتها من الذاكرة. تُخزن كائنات ProductResponse (وليس كيانات JPA) بمدة صلاحية 60 ثانية، ويتم إبطال التخزين عند أي تغيير في المخزون حتى لا تُقدّم بيانات قديمة.

```
@Cacheable(value = "products", key = "'all'") //
Redis قراءة: تُخدّم من
public List<ProductResponse> findAll() { ... }

@CacheEvict(value = "products", allEntries = true) //
كتابة: تُبطل التخزين
public Product create(Product product) { ... }
```

يتم التحكم بالتخزين عبر الإعداد `spring.cache.type`: القيمة `redis` تُفعّله، والقيمة `none` تعطله. هذا يسمح بتشغيل نفس صورة التطبيق في الحالتين «قبل» و«بعد» — لا يتغيّر سوى الإعداد، مما يضمن مقارنة عادلة.

8.2 تصميم تجربة القياس (قبل / بعد)

باستخدام أداة الضغط نفسها (JMeter)، تم توليد جمل قراءة على كتالوج كبير (5000 منتج لجعل القراءة «مكلفة» فعلاً) مرتين متتاليتين:

- قبل (Cache OFF): `spring.cache.type=none` — كل طلب يصل إلى قاعدة البيانات.
- بعد (Cache ON): `spring.cache.type=redis` — الطلبات المتكررة تُخدّم من Redis.

يتم تشغيل التجربة بأمر واحد: `./loadtest/bench.sh cache/`.

8.3 النتائج وتحليل عنق الزجاجة

أظهرت المقارنة تحسناً كبيراً عند تفعيل التخزين المؤقت — خاصةً في ذيل التوزيع (p95/p99) وفي عدد الطلبات في الثانية، مع اختفاء الأخطاء الناتجة عن إشباع النظام. الأرقام الدقيقة تظهر في الشكلين أدناه.

أولاً: النتيجة من الطرفية (Terminal) — جدول قبل/بعد

```
RESULT - Req 6/10 caching before vs after

=== big before ===
GET /api/products (hot read path) n= 800 err%= 0.0 median= 186 p95= 307 p99= 348 mean= 181 tps= 17.3 (ms)

=== big after ===
GET /api/products (hot read path) n= 800 err%= 0.0 median= 163 p95= 299 p99= 694 mean= 167 tps= 17.4 (ms)

Raw results: loadtest/results/big_before.jtl vs big_after.jtl
```

شكل 15: مخرجات `bench.sh cache` — مقارنة زمن الاستجابة (p95/p99) وعدد الطلبات/ثانية ونسبة الأخطاء، بدون تخزين مقابل مع التخزين

ثانياً: تقرير JMeter — قبل التخزين المؤقت (Cache OFF)

Statistics															
Requests		Executions			Response Times (ms)							Throughput		Network (KB/sec)	
Label	#	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent	
Total		800	0	0.00%	180.64	38	405	186.00	282.90	307.00	348.99	17.26	30988.25	2.60	
GET /api/products (hot read path)		800	0	0.00%	180.64	38	405	186.00	282.90	307.00	348.99	17.26	30988.25	2.60	

شكل 16: لوحة JMeter بدون تخزين مؤقت — أزمدة استجابة مرتفعة في الذيل (p95/p99) وعدد طلبات/ثانية منخفض

ثالثاً: تقرير JMeter — بعد التخزين المؤقت (Cache ON)

Statistics															
Requests		Executions			Response Times (ms)							Throughput		Network (KB/sec)	
Label	~	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent	
Total		800	0	0.00%	167.22	33	965	163.00	269.90	299.00	724.69	17.43	31288.65	2.62	
GET /api/products (hot read path)		800	0	0.00%	167.22	33	965	163.00	269.90	299.00	724.69	17.43	31288.65	2.62	

شكل 17: لوحة JMeter مع التخزين المؤقت — انخفاض كبير في p95/p99 وارتفاع واضح في عدد الطلبات/ثانية

تحليل عنق الزجاجة (المتطلب 10): العائق الحقيقي هو مجمع اتصالات قاعدة البيانات (HikariCP، الحد الأقصى = 10). تحت الحمل العالي، تتنافس القراءات غير المخزنة على هذه الاتصالات العشرة فتتراكم الطلبات وتنفجر أزمدة الاستجابة في الذيل (p99). عند تفعيل Redis، تُخدم القراءات المتكررة من الذاكرة وتتحزّر اتصالات قاعدة البيانات، فينخفض p99 بشكل حادّ ويرتفع عدد الطلبات في الثانية. هذا بالضبط هو تحديد عنق الزجاجة وإثبات أثر الحل عليه.

9. المعاملات الذرية (ACID Transactions) — الذرية في إنشاء الطلب

إنشاء الطلب عملية مركبة: خصم المخزون لعدة منتجات، ثم إنشاء الطلب، ثم إنشاء عناصره. يجب أن تتم هذه الخطوات بالكامل أو لا تتم إطلاقاً — وهذه هي خاصية الذرية (Atomicity) من خصائص قواعد بيانات ACID. تتحقق هذه الخاصية عبر التعليق التوضيحي @Transactional الذي يجعل كل ما بداخل الدالة معاملةً واحدة.

```
@Transactional // كل ما بالداخل معاملة واحدة - ذرية
public Order placeOrder(UUID userId, PlaceOrderRequest request)
{
    User user = resolveUser(userId);
    List<OrderItem> items = new ArrayList<>();

    for (var itemReq : request.getItems()) {
        // خصم المخزون مع القفل المتفائل - قد يرمي استثناءً إن
        // نفذ المخزون
        Product product = optimisticStrategy.purchase(
            itemReq.getProductId(), itemReq.getQuantity());
        items.add(/* ... عنصر الطلب ... */);
    }

    Order order = orderRepository.save(/* ... الطلب ... */);
    order.getItems().addAll(items);
    orderRepository.save(order);

    eventPublisher.publishEvent(new OrderPlacedEvent(this,
        order)); // فوتر غير متزامنة
    return order;
} // أي استثناء بالأعلى =>
(Rollback) تراجع كامل
```

مثال على الأهمية: لو طلب العميل ثلاثة منتجات ونفذ مخزون الثالث، فبدون المعاملة كان سيُخصم مخزون أوّل منتجين ثم تفشل العملية — تاركةً بيانات غير متّسقة (مخزون نقص دون طلب مقابل). مع @Transactional يُلغى كل شيء تلقائياً: لا يُنشأ طلب، ولا يُخصم أيّ مخزون.

هذه الذرية — مجتمعةً مع آليات القفل (المتطلب 7، الموصّحة في القسم 2) — هي ما يضمن النتيجة التي رأيناها في القسم 7: كل طلب ناجح متّسق تماماً، دون بيع زائد أو فقدان بيانات تحت التزامن العالي.

10. ملخص الملفات والتقنيات

المتطلب	التقنية / النمط	الملفات الرئيسية
1. حماية البيانات من التضارب	Optimistic Locking (@Version) Pessimistic Locking (SELECT FOR UPDATE) Strategy Pattern	Product.java OptimisticPurchaseStrategy.java PessimisticPurchaseStrategy.java FlashSaleRepository.java
2. إدارة الموارد الحاسوبية	HikariCP Connection Pool Lettuce Redis Pool ThreadPoolTaskExecutor CallerRunsPolicy	application.yml AsyncConfig.java
3. المعالجة غير المتزامنة	Async + @EventListener@ Spring ApplicationEvent Observer Pattern	OrderPlacedEvent.java OrderEventListener.java InvoiceService.java EmailService.java
4. معالجة البيانات على دفعات	Scheduled (cron)@ Pageable Chunk Processing Idempotent execution	DailySalesJob.java DailySalesReportService.java TokenCleanupScheduler.java
5. توزيع الأحمال	Nginx Round-Robin Docker Compose scaling AOP Request Logging Prometheus + Grafana	docker-compose.yml nginx/nginx.conf RequestLoggingAspect.java */monitoring
اختبار الضغط (Stress Testing)	Apache JMeter (Docker) 100 مستخدم متزامن التحقق من سلامة البيانات	loadtest/bench.sh loadtest/shopflow-writeonly.jmx
6 + 10. التخزين المؤقت والقياس المرجعي	Redis Cache (@Cacheable / @CacheEvict) قياس قبل/بعد تحليل عنق الزجاجة (HikariCP)	CacheConfig.java ProductService.java loadtest/bench.sh loadtest/shopflow-readonly.jmx
7. القفل المتفائل والمتشائم	Optimistic Locking (@Version) Pessimistic Locking (SELECT FOR UPDATE) (مشروح في القسم 2)	Product.java OptimisticPurchaseStrategy.java PessimisticPurchaseStrategy.java

OrderService.java	Transactional@ الذرية (Atomicity) والتراجع التلقائي (Rollback)	8. المعاملات الذرية (ACID)
-------------------	--	-------------------------------